

Текст для устной презентации

«Виды серверного ПО: установка и обслуживание»

Слайд 1 — Титульный

Добрый день! Сегодня мы поговорим о серверном программном обеспечении — его видах, о том, как правильно его устанавливать и как обслуживать, чтобы сервера работали стабильно и надёжно. Эта тема актуальна для любого, кто работает в сфере IT или администрирования систем.

Слайд 2 — План презентации

Наша презентация разбита на пять блоков.

Сначала мы разберёмся, что вообще такое серверное ПО. Затем рассмотрим его основные виды. Потом перейдём к установке — с конкретными шагами и командами. После этого поговорим об обслуживании: обновлениях, безопасности, резервном копировании. И в конце — лучшие практики, которые помогут поддерживать сервера в хорошем состоянии.

Слайд 3 — Что такое серверное ПО

Серверное программное обеспечение — это ПО, которое работает на выделенном компьютере или виртуальной машине, обрабатывает запросы от других компьютеров в сети и предоставляет им какие-то услуги.

Простой пример: когда вы открываете сайт в браузере, браузер отправляет запрос, а веб-сервер — специальная программа на удалённом компьютере — обрабатывает этот запрос и возвращает вам страницу.

Три ключевых особенности серверного ПО: оно работает по архитектуре «клиент–сервер», то есть один сервер обслуживает множество клиентов одновременно. Оно рассчитано на непрерывную работу — 24 часа в сутки, 7 дней в неделю. И оно должно эффективно управлять ресурсами: процессором, памятью, сетью,

дисками.

Слайд 4 — Виды серверного ПО

Давайте пройдемся по основным видам. Их несколько:

Веб-серверы — принимают запросы браузеров и отдают HTML-страницы. Самые известные — Apache, Nginx и IIS от Microsoft.

Серверы баз данных — хранят и обрабатывают данные приложений. MySQL, PostgreSQL — для открытых решений, Microsoft SQL Server — для корпоративной среды Windows.

Файловые серверы — организуют общий доступ к файлам по сети. Samba позволяет Linux-серверу работать в Windows-сети, NFS — стандарт для Unix-систем.

Почтовые серверы — Postfix и Sendmail на Linux, Microsoft Exchange на Windows.

DNS и DHCP — без них сеть просто не работает. DNS переводит доменные имена в IP-адреса, DHCP автоматически раздаёт IP-адреса устройствам.

Серверы приложений — запускают бизнес-логику: Tomcat для Java, Node.js для JavaScript, JBoss для enterprise-приложений.

Слайд 5 — Веб-серверы и серверы БД (подробнее)

Остановимся чуть подробнее на двух самых популярных категориях.

Веб-серверы:

- Apache — наиболее распространён, около 30% рынка. Удобен модульной архитектурой, легко расширяется.
- Nginx произносится «Энджинкс». Отлично справляется с большим количеством одновременных подключений, часто используется как обратный прокси перед другими серверами.
- IIS — встроен в Windows Server, хорошо интегрирован с .NET.

Серверы БД:

- MySQL и MariaDB (её форк) — стандарт для большинства веб-проектов.

Быстрые, простые в настройке.

- PostgreSQL — более мощная система с поддержкой сложных запросов, транзакций, расширений. Хороший выбор для серьёзных проектов.
 - MongoDB — если данные неструктурированные или меняются по схеме, NoSQL-решения вроде MongoDB будут удобнее.
-

Слайд 6 — Установка серверного ПО: шаги

Теперь поговорим об установке. Это не просто «скачал и запустил» — это процесс, который нужно делать правильно.

Шаг 1 — Планирование. Прежде всего определяем: какая ОС будет, сколько нужно RAM и CPU, какой объём диска. Ubuntu Server — хороший выбор для большинства задач. CentOS/Rocky Linux — для корпоративной среды. Windows Server — если нужна интеграция с Active Directory.

Шаг 2 — Подготовка среды. Обновляем ОС, задаём правильный hostname, настраиваем firewall. Создаём пользователей с нужными, но не избыточными правами.

Шаг 3 — Установка пакетов. На Linux используем менеджеры пакетов: `apt` на Ubuntu/Debian, `yum` на CentOS. Они автоматически загружают нужные зависимости.

Шаг 4 — Конфигурация. Это самый важный шаг. Редактируем конфигурационные файлы: задаём порты, пути, лимиты соединений, настраиваем SSL.

Шаг 5 — Тестирование. Запускаем сервис, проверяем логи, убеждаемся что порт открыт, выполняем базовые тесты.

Слайд 7 — Примеры команд

На этом слайде — конкретные команды, которые используются при установке.

На Ubuntu/Debian установка Nginx выглядит так: обновляем список пакетов, устанавливаем `nginx`, включаем его в автозапуск командой `systemctl enable` и запускаем.

MySQL устанавливается аналогично, но после установки обязательно запускаем скрипт `mysql_secure_installation`, который помогает закрыть базовые уязвимости:

убирает анонимные аккаунты, устанавливает пароль root.

На Windows Server используем PowerShell. Роль IIS устанавливается командой `Install-WindowsFeature`. Управление службами — через `Get-Service`, `Start-Service`, `Stop-Service`.

Слайд 8 — Обслуживание и мониторинг

Установить сервер — это только половина дела. Дальше начинается регулярное обслуживание.

Обновления — самое важное. Устаревшее ПО = дыры в безопасности. Но обновления нельзя применять сразу в production — сначала тестируем на копии. Ведём журнал изменений.

Мониторинг — нужно знать, что происходит на сервере. Следим за загрузкой CPU и памяти, за местом на диске, за ошибками в логах. Zabbix и Nagios — популярные open-source решения. Grafana + Prometheus — для красивых дашбордов с метриками в реальном времени.

Резервное копирование — правило простое: если бэкапа нет — данных нет. Делаем ежедневно, храним в нескольких местах (локально + облако). И главное — периодически проверяем, что бэкапы реально восстанавливаются.

Безопасность — закрываем все ненужные порты, используем fail2ban для защиты от brute-force атак, настраиваем SSL-сертификаты. Let's Encrypt даёт бесплатные сертификаты для публичных серверов.

Слайд 9 — Популярные инструменты

Несколько слов о конкретных инструментах:

Zabbix — наверное, самая популярная open-source система мониторинга в России. Умеет следить за серверами, сетевым оборудованием, базами данных, снимать десятки метрик.

Ansible — позволяет управлять конфигурацией серверов через YAML-файлы (playbooks). Не требует агентов, работает через SSH. Очень удобен когда серверов много.

Docker — контейнеризация упрощает установку и обновление ПО. Вместо «настройки на голом железе» запускаем готовый контейнер. Обновление = замена

контейнера.

Fail2ban — анализирует логи и автоматически блокирует IP-адреса, с которых идут попытки перебора паролей.

Слайд 10 — Лучшие практики

Перед тем как завершить — несколько правил, которые я бы назвал «заповедями» администратора.

1. **Документируйте всё.** Любое изменение в конфигурации должно быть записано: что, когда, зачем изменили.
 2. **Принцип минимальных привилегий.** Nginx не должен работать от root. Каждый сервис — под своим пользователем с минимально необходимыми правами.
 3. **Тестовая среда.** Никогда не применяйте изменения сразу в production. Сначала проверьте на staging-сервере.
 4. **Автоматизируйте рутину.** Скрипты, Ansible, cron — экономят время и исключают человеческий фактор.
 5. **Регулярный аудит безопасности.** Раз в месяц проверяем: нет ли устаревших пакетов, не открылись ли лишние порты.
 6. **Планирование мощности.** Следим за трендами нагрузки. Гораздо лучше добавить ресурсы заранее, чем в авральном режиме во время инцидента.
-

Слайд 11 — Итоги

Давайте подведём итог.

Серверное ПО — это основа любой IT-инфраструктуры. Без правильно настроенных серверов не работают ни сайты, ни корпоративная почта, ни файловые хранилища.

Выбор конкретного вида серверного ПО всегда определяется задачей: для веба — Apache или Nginx, для хранения данных — PostgreSQL или MySQL, для почты — Postfix или Exchange.

Правильная установка — это не только запуск сервиса, но и документация, настройка безопасности, тестирование.

Обслуживание — это непрерывный процесс: обновления, мониторинг, бэкапы.

И наконец, автоматизация и мониторинг — это то, что отличает профессионального администратора от начинающего.

Спасибо за внимание! Готов ответить на ваши вопросы.

Приблизительное время презентации: 15–20 минут